

HAZOMAT

B39VS - Systems Project

Abdul Maajid Aga - H00431236

Muhammad Abban - H00427641

Laith Samir Mohamed - H00427630

Aisultan Alpysbay - H00407970

Moaiz Saeed - H00451062

Sufyaan Atif - H00434736

School of Engineering and Physical Sciences
Heriot-Watt University Dubai

April 2025

Abstract

Meow meow

Contents

1	Introduction	3
1.1	Project overview	3
1.2	Aims and objectives	3
1.3	Roles and Responsibilities	4
1.4	Project Scope	7
2	Background and Theory	8
2.1	Literature review	8
2.2	Motor Control	8
2.2.1	Mecanum wheels	8
2.2.2	Wheel encoders	8
2.2.3	PID control	9
2.2.4	PWM control	10
2.3	Sensing	10
2.3.1	Odometry	10
2.3.2	Localisation and Mapping	11
3	Methodology and Implementation	11
3.1	Concept Development and Evaluation	12
3.2	Digital Design	13
3.3	Mechanical Design	13
3.3.1	Mobile Base	13
3.3.2	Manipulator	13
3.3.3	Assembly	13
3.4	Sensors	13
3.5	Software	13
3.5.1	SLAM	14
3.5.2	Vision	16
3.5.3	Embedded systems	16
3.5.4	Control	17
3.5.5	Audio feedback	18
3.6	Circuit schematics	18
3.7	Prototype development	20
4	Results	20
4.1	Computer simulation	20
4.2	Testing results	20
5	Management and Marketing	22
5.1	Project Management	22

5.2	Project Finance	22
5.3	Marketing	22
5.4	Risk Management	22
5.5	Health, Safety, and Environment	22
6	Conclusion and Future Work	22
	References	22

1 Introduction

1.1 Project overview

The project involves the design and development of an autonomous mobile manipulator capable of picking up waste from waste points and disposing of it at the base station.

The robot, designated 'Hazmat' manufactured by SLAM Industries, is equipped with a Raspberry Pi 5 as its main processor, assisted by an ESP32-S3 to offload computational tasks. This configuration reduces the processing burden on the Pi and provides an additional safeguard against potential hardware faults due to faulty wiring.

Hazmat is fitted with mecanum wheels, enabling omni-directional movement and allowing it to smoothly traverse through tight spaces within a factory environment. The robot utilizes a stereo-depth camera (Oak-D Pro W) with an integrated IMU, motor encoders, and a 2D LiDAR (RpLiDAR A3) to perform simultaneous localization and mapping (SLAM), enabling it to autonomously locate waste containers and navigate to disposal points.

Hazmat is also equipped with a 3-DOF manipulator designed using a four-bar linkage mechanism, and a dual-claw soft servo gripper for efficient waste pickup and placement. The servo motors were modified to output potentiometer feedback to the Pi, enabling real-time inverse kinematics calculations for the robotic arm. Power is supplied by a lithium polymer battery that drives the wheels, while a buck converter is used to step down voltage for the robot's electronic systems.

1.2 Aims and objectives

Aims:

- To design, fabricate, and demonstrate a functional, autonomous mobile manipulator capable of navigating an indoor environment, picking up hazardous waste containers, and delivering them to designated disposal areas with minimal human intervention.

Objectives:

- Design and fabricate a stable mobile robot based on mecanum wheels to allow omnidirectional movement in constrained indoor environments.
- Develop a secure container compartment to safely transport waste containers during robot motion.
- Integrate vision and ranging sensors (stereo camera, 2D LiDAR, ultrasonic sensors) to enable environmental perception, obstacle detection, and navigation.
- Design and build a 3-DOF robotic manipulator with a four-bar linkage for grasping and moving waste containers.
- Implement autonomous navigation using SLAM (Simultaneous Localization and Mapping) and obstacle avoidance algorithms.
- Implement object and container recognition using colour coding and computer vision methods.
- Develop audio feedback features to alert humans during path obstruction scenarios.
- Integrate micro-ROS middleware to allow reliable communication between the embedded control systems and the central processor.
- Implement a user interface and visual indicators (LEDs or screen) to display robot status and task progress.
- Develop a simple autonomous charging strategy (proposal stage) for future continuous operation.
- Simulate and test robot behaviour in Gazebo or a relevant simulation environment before real-world testing.
- Manage team roles and collaborate effectively across mechanical, electrical, and software domains to achieve system integration.

1.3 Roles and Responsibilities

SLAM Industries started as a military contractor, developing advanced autonomous systems for extreme environments. Our team created robots for reconnaissance, high-risk operations, and more, pushing the limits of robotics technology. Now, we have shifted focus, using our military-grade expertise to design robots for a wide range of industries. From hazardous waste management to autonomous transportation, we deliver reliable, high-performance solutions.

SLAM Industries is a team of six passionate individuals committed to redefining the future of autonomous systems. The company structure is organized as shown in Figure 1, and the key responsibilities and contributions are as follows:

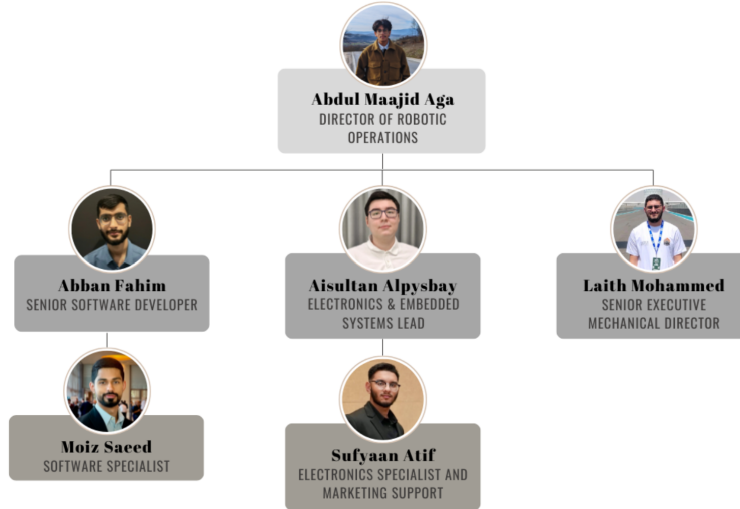


Figure 1: Organisation hierarchy

Abdul Maajid Aga – Director of Robotic Operations

- Designed the robot base, top plate, and structural stands, ensuring organized component layout and accessibility.
- Co-developed the 3-DOF four-bar linkage arm with Laith.
- Migrated wheel control from serial to micro-ROS on the ESP32-S3.
- Helped wire and verify the full electrical system.
- Developed inverse kinematics algorithms for the arm (not used in final version).
- Created and refined early prototypes for mechanical testing.
- Optimized the robot's structure for easy assembly, maintenance, and manufacturing.
- 3D-printed and assembled mechanical components.
- Led the team, assigned tasks, enforced timelines, and maintained momentum.
- Provided morale support and ensured teammate well-being throughout the project.

Laith Mohammed – Senior Executive Mechanical Director

- Assisted in designing and prototyping multiple robot base plates using laser-cut acrylic until finalizing the design.
- Designed and prototyped several robotic arm versions using laser-cut acrylic to reach the final version.
- Sourced power converters and built cable harnesses for power distribution.
- Assembled a custom PCB using a perf board to securely integrate the ESP32 with the motor driver board.
- Developed a C++ library to control and read the speed of the mecanum wheels.
- Coordinated between electrical, mechanical, and software teams.

Aisultan Alpysbay – Electronics and Embedded Systems Lead

- Designed the electrical schematic, simplified diagram, and initial PCB (not used in final system).
- Assembled the robotic arm multiple times, including rebuilds after failures.
- Rewired the robot's electronics twice for improved reliability.
- Replaced damaged motor drivers.
- Researched and provided design inspiration for the final robot layout.
- Soldered additional wiring onto four servos for sensor feedback (one later replaced).
- Suggested vision-based alignment techniques to improve arm positioning.

Sufyaan Atif – Electronics Specialist and Marketing Support

- Designed the complete electrical schematic and simplified system power flow diagram.
- Gathered and organized all required electronic components.
- Assembled and wired the robot's electrical system.
- Tested DC motors, encoders, and sensors for functionality.
- Wrote code to read encoder pulses and calculate wheel RPM.
- Calibrated encoders for reliable measurements.
- Assembled the robotic arm and programmed servo operating limits.
- Tested arm movement with different joint angles.
- Managed the project website and marketing activities.

Abban Fahim – Senior Software Developer

- Interfaced sensors with Raspberry Pi, including LiDAR and stereo camera.

- Set up the complete ROS framework, launch scripts, and development environment on Raspberry Pi.
- Implemented inverse kinematics and PID controller for mobile base control.
- Created a simulation environment to test mobile robot behavior.
- Assisted in implementing SLAM and color detection nodes.
- Developed Aruco marker detection and localization system.
- Built the autonomous navigation stack, fusing multi-sensor data and odometry.
- Wrote the initial serial communication prototype between Pi and ESP.
- Programmed final I2C communication system for the manipulator.
- Mentored the team on Linux, version control, and ROS fundamentals.

Moiz Saeed – Software Intern

- Assisted in designing the robot's base structure.
- Developed and tested the odometry ROS node.
- Supported the implementation of Cartographer SLAM using LiDAR for mapping and localization.
- Wrote the full URDF (Unified Robot Description Format) model for the robot.
- Helped set up the robot's transformation (TF) frames in ROS.
- Implemented a colour detection system using OpenCV.
- Developed a ROS node for audio feedback and notifications.
- Assisted in the development of the robot's autonomous navigation system.

1.4 Project Scope

The scope of project Hazmat is to design, develop, and deploy an autonomous mobile manipulator capable of detecting, navigating to, and safely handling hazardous materials in indoor industrial environments. The system integrates a mecanum-wheeled base for omnidirectional mobility, a 3-DOF robotic arm with a four-bar linkage for stable manipulation, and a multi-sensor perception suite comprising ultrasonic sensors, an RPLiDAR C1, and an Oak-D depth camera for SLAM and object recognition.

The project includes:

- Development of motion control algorithms for precise omnidirectional navigation.

- Implementation of object detection and tracking for hazardous item identification.
- Real-time SLAM integration using LiDAR and depth vision for autonomous localization and mapping.
- Robotic arm control with feedback for precise grasping and placement.
- ROS-based communication framework across subsystems using micro-ROS and a Raspberry Pi 5.
- Power management and autonomy optimization for operation within a 2+ hour runtime window.

Out of Scope:

- Outdoor navigation and GPS-based localization.
- Handling of liquid or airborne hazardous materials.
- Integration with cloud-based monitoring or remote control systems.

2 Background and Theory

2.1 Literature review

2.2 Motor Control

2.2.1 Mecanum wheels

Mecanum wheels allow a robot to move in any direction without changing its orientation. Each wheel has angled rollers (at 45°) mounted around its rim. When the wheels spin, they create a combination of forward/backward movement and sideways movement. By carefully controlling the speed and direction of each wheel, the robot can move forward, backward, sideways (strafe), diagonally, or rotate on the spot. This gives the robot full omnidirectional mobility, as outlined in Figure 2.

2.2.2 Wheel encoders

The mecanum wheels come along with an encoder for each wheel and the encoders operate through hall effect sensors. A hall effect sensor works by detecting the magnetic field from a magnet ring attached to the wheel shaft. As the wheel turns, the magnet ring spins with it, causing the magnetic field near the hall sensor to change. Every time the sensor detects a change in the magnetic field, it creates an electrical pulse. These pulses are then sent to the microcontroller to find out the wheel RPM.

To calculate the RPM, the motor's encoder pulses were counted over a fixed period. Each full revolution generates a known number of pulses (20 pulses per revolution in our motor). The pulse count was divided by the number of

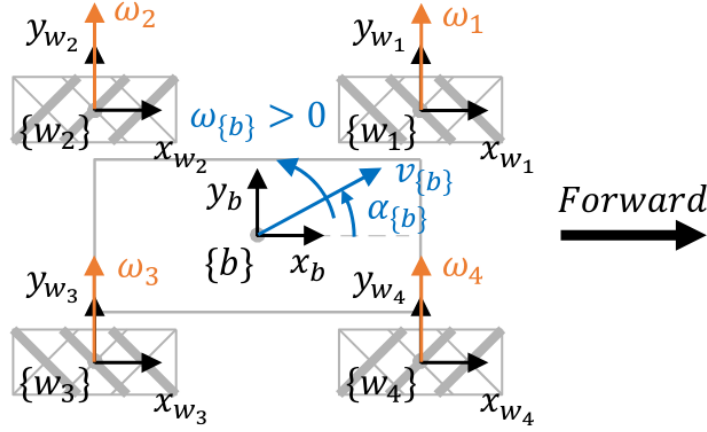


Figure 2: A common 4-wheeled mecanum configuration

pulses per revolution to determine the number of revolutions per second and then multiplied by 60 to convert it into revolutions per minute (RPM). After each calculation, the pulse count was reset for accurate measurement in the next cycle.

2.2.3 PID control

A PID controller (Proportional-Integral-Derivative controller) is used in a closed-loop feedback system to control the outer loop of the robot. In this system, the robot's current joint position is continuously measured. The error is calculated by comparing the current position to the desired position. The PID controller then processes this error to generate a control input that tries to minimize it: the proportional (P) part reacts to the size of the error, the integral (I) part reacts to how long the error has been there (accumulated error over time), and the derivative (D) part reacts to how fast the error is changing. The PID controller continuously adjusts the robot's movement based on this feedback, aiming to keep it closely following the target trajectory. The following equations describe this controller mathematically, and Figure 3 outlines it as a block diagram,

$$e = r - y$$

$$u = k_P e + k_I \int_0^t e \, dt + k_D \dot{e}$$

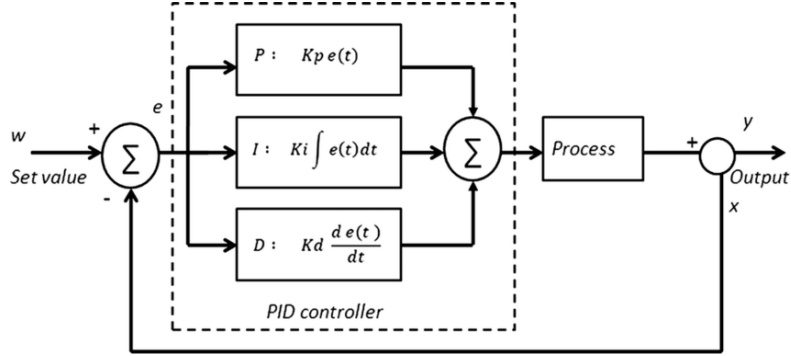


Figure 3: Block diagram of PID control

2.2.4 PWM control

2.3 Sensing

2.3.1 Odometry

Odometry is the process of estimating a robot's change in position (x, y) and orientation (θ) by integrating its measured velocities over time. In wheeled robots, we typically derive those velocities from the wheel encoders' readings i.e. RPM values in our case.

The process of computing the robot's motion begins with the acquisition of RPM values from the wheels. These RPM values are first converted into angular velocities to represent the rotational speed of each wheel in radians per second. Once the angular velocities ($\omega_{fl}, \omega_{fr}, \omega_{rl}, \omega_{rr}$) are obtained, the forward kinematics equations are applied to determine the robot's linear velocities in the x and y directions, as well as its angular velocity about the z-axis, given the radius of the wheels r , and length l_x and width l_y from the 2D centre of the robot

$$v_x = \frac{r}{4}(\omega_{fl} + \omega_{fr} + \omega_{rl} + \omega_{rr})$$

$$v_y = \frac{r}{4}(-\omega_{fl} + \omega_{fr} + \omega_{rl} - \omega_{rr})$$

$$\omega_z = \frac{r}{4(l_x + l_y)}(-\omega_{fl} + \omega_{fr} - \omega_{rl} + \omega_{rr})$$

Following the computation of the velocities, integration over time is performed to update the robot's position and orientation. This integration yields the robot's displacement along the x and y, and orientation along z, effectively estimating its trajectory over time based on the measured motion data

$$\begin{aligned}
x &= \int v_x dt \\
y &= \int v_y dt \\
\theta &= \int \omega_z dt
\end{aligned}$$

2.3.2 Localisation and Mapping

Localisation is a fundamental ability required by robots for autonomous navigation, as it involves estimation of the robots pose (position and orientation) in the same frame of reference as its goal. The other component is mapping, which attempts to create a map of the obstacles and areas of interest to the robot, which is crucial to allow for path planning with obstacle avoidance. This gives way to the common problem tackled in robotics known as simultaneous localisation and mapping (SLAM), which attempts to approximate both in a computationally feasible ways. Since all quantities in this problem are stochastic and discrete, the objective is to compute the distribution of the robot state, x_t , the map of the environment, m_t , given a series of observations, $o_{1:t}$, and control inputs, $u_{1:t}$,

$$P(m_{t+1}, x_{t+1} | o_{1:t+1}, u_t)$$

Using Bayesian computation, the posterior distributions of each x_t and m_t can be calculated separately when given the value of the other. Many statistical algorithms exist to tackle this problem, like the Extended Kalman Filter and sequential Monte Carlo method, as well as graph-based methods. Amongst these the sequential Monte Carlo method, also known as the particle filter, is most popular in mobile robots. It models the states and observations as state-space equations given respectively as

$$\begin{aligned}
x_t &= g(x_{t-1}) + W_{t-1} \\
o_t &= h(x_t) + V_t
\end{aligned}$$

where W_t and V_t are mutually independent, and the equations are sequentially solved using a number of different Bayesian methods depending on the constraints of the specific system.

3 Methodology and Implementation

In the current era of rapid technological advancement, the shift toward autonomous systems has become not only logical but essential. Automation reduces human risk exposure, optimizes operational efficiency, and expands capabilities beyond biological limitations. Tasks involving hazardous materials

present significant health and safety concerns for human operators. Exposure to toxic compounds, unstable environments, and unpredictable variables necessitates the development of non-human agents capable of executing such operations with precision and resilience.

This chapter outlines the design methodology and implementation sequence of an autonomous hazardous waste retrieval unit. The system has been engineered for deployment in environments deemed unsafe for prolonged human presence. Operational goals include waste identification, secure collection, and optimized navigation, all executed without direct human intervention.

3.1 Concept Development and Evaluation

Our primary novel concept was the manipulator, hence we considered multiple designs, and used a design choice matrix to find the most appropriate one. Our criterion were: the appropriateness and the end-effector to our task and its simplicity; the accuracy that could be achieved in end-effector positions; the static and dynamic (during movement of joints or mobile base) stability of the manipulator; the load it could carry; and finally the complexity of its implementation.

The designs we considered as follow, and their decision matrix scores are shown in Table 1,

- A 2-DOF forklift that would require objects to be placed at some elevation and would function with two linear actuators that would move the hoist horizontally in the direction of the robots movement and vertically up or down.
- A suction gripper that would be limited to working on flat surfaces, and would have three revolute joints for a full range of motion in 3D space.
- A crane mechanism with one rotating joint around the z (vertical) axis and one linear actuator to lower or raise the hoist to be able to reach and hold the objects in a safe place.
- A simple 3-DOF arm with only revolute joints and a two-claw grippers was chosen as a baseline for the rest. It concluded as the best choice and was further developed with a two-bar linkage mechanism to further improve stability by bring the manipulator's centre of the mass lower.

Gripper type	Weight	Forklift	Suction gripper	Crane	Regular 3-DOF arm
Degrees of freedom	0	2	3	3	3
End effector	25	1	2	2	3
Accuracy/tolerance	25	5	4	3	3
Stability & weight	25	1	3	2	3
Strength & load capacity	15	5	1	3	3
Electrical & programming complexity	10	4	3	3	3
Final score	0	290	270	250	300

3.2 Digital Design

3.3 Mechanical Design

3.3.1 Mobile Base

3.3.2 Manipulator

3.3.3 Assembly

3.4 Sensors

3.5 Software

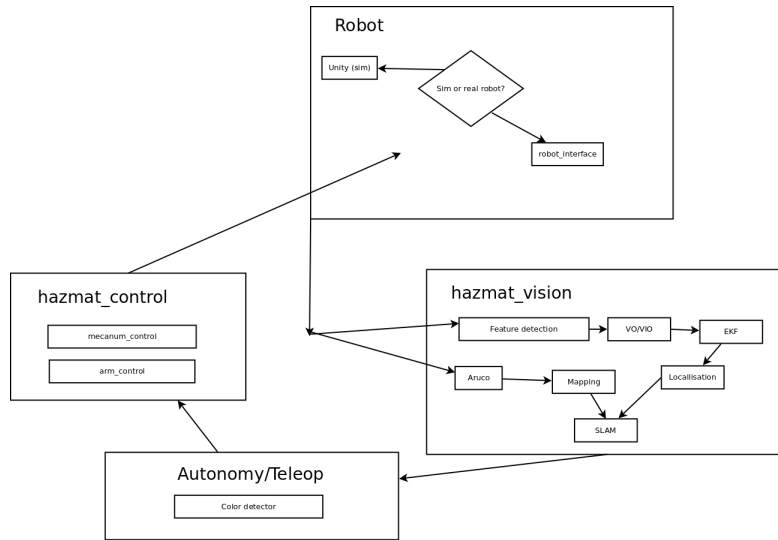


Figure 4: High level overview of the ROS nodes and topics

Our software stack was primarily built upon the expansive ecosystem of

Robot Operating System (ROS) [1], which provided cross-script communication, parallelisation, and ready-to-use packages maintained by robotics companies and industry experts, out of the box. Figure 4

3.5.1 SLAM

The robots transform tree was carefully structured in the Universal Robot Description File (URDF) to support Cartographer’s configurations. The IMU ([imu_link](#)) and LiDAR ([scan_link](#)) were each fixed relative to the central robot frame ([base_link](#)). The overall Transformation (TF) hierarchy followed the structure:

Our SLAM solution was built upon the ROS packages for Google’s Cartographer library, configured for 2D mapping using LiDAR and IMU data. The core configuration for Cartographer was defined in the Lua script (`cartographer.lua`), where key parameters such as `use_trajectory_builder_2d = true` and `use_imu_data = true` were set to enable 2D SLAM operation and integrate IMU data into the state estimation pipeline. The IMU played a crucial role in providing continuous orientation and rotational velocity data, improving pose estimation accuracy, particularly during rapid movements or in environments with sparse LiDAR features. The following listing is our lua configuration file for Cartographer:

```
include "map_builder.lua"
include "trajectory_builder.lua"

options = {
  map_builder = MAP_BUILDER,
  trajectory_builder = TRAJECTORY_BUILDER,
  map_frame = "map",
  tracking_frame = "imu_link",
  published_frame = "imu_link",
  odom_frame = "odom",
  provide_odom_frame = true,
  publish_frame_projected_to_2d = true,
  use_odometry = false,
  use_nav_sat = false,
  use_landmarks = false,
  publish_to_tf = true,
  num_laser_scans = 1,
  num_multi_echo_laser_scans = 0,
  num_subdivisions_per_laser_scan = 1,
  num_point_clouds = 0,
  lookup_transform_timeout_sec = 0.2,
  submap_publish_period_sec = 0.3,
  pose_publish_period_sec = 5e-3,
  trajectory_publish_period_sec = 30e-3,
```

```

    rangefinder_sampling_ratio = 1.,
    odometry_sampling_ratio = 1.,
    fixed_frame_pose_sampling_ratio = 1.,
    imu_sampling_ratio = 1.,
    landmarks_sampling_ratio = 1.,
}

MAP_BUILDER.use_trajectory_builder_2d = true

TRAJECTORY_BUILDER_2D.min_range = 0.05
TRAJECTORY_BUILDER_2D.max_range = 3.
TRAJECTORY_BUILDER_2D.missing_data_ray_length = 3.
TRAJECTORY_BUILDER_2D.use_imu_data = true
TRAJECTORY_BUILDER_2D.use_online_correlative_scan_matching = true
TRAJECTORY_BUILDER_2D.motion_filter.max_angle_radians = math.rad(0.1)

POSE_GRAPH.constraint_builder.min_score = 0.65
POSE_GRAPH.constraint_builder.global_localization_min_score = 0.7

```

The LiDAR sensor was responsible for generating 2D laser scans of the surrounding environment. These scans were pre-processed through a custom filtering step implemented in the `scan_callback()` function of the Color node. In this filtering process, a forward-facing sector was isolated (defined between -80° and $+100^\circ$) and invalid or noisy range readings were discarded based on `range_min` and `range_max` conditions. The filtered data was published on the `/scan_filtered` topic, which was then used by the Cartographer SLAM node for map building. Cartographer constructed an occupancy grid map by matching these incoming filtered scans against the existing map. The following listing shows the code for `scan_callback`:

```

def scan_callback(self, msg: LaserScan):
    scan = msg
    self.scan_data = msg
    # Define the front sector in radians
    front_sector_min = -math.radians(80) # -0.5236 rad
    front_sector_max = math.radians(100) # 0.5236 rad

    # Calculate the index range corresponding to the front sector
    # LaserScan angles: angle = scan.angle_min + index * scan.angle_increment
    start_index = int((front_sector_min - scan.angle_min) / scan.angle_increment)
    end_index = int((front_sector_max - scan.angle_min) / scan.angle_increment)

    # Ensure indices are within valid bounds
    start_index = max(0, start_index)
    end_index = min(len(scan.ranges) - 1, end_index)

```

```

# Extract the ranges within the desired sector and filter valid readings
front_ranges = [r for r in scan.ranges[start_index:end_index+1]
if scan.range_min <= r <= scan.range_max]:
    if front_ranges:
        self.closest_distance = min(front_ranges)
    else:
        print("No valid range measurements in the front sector.")

```

To further enhance localization robustness, `use_online_correlative_scan_matching` was enabled in the Lua configuration, and motion filter parameters such as `motion_filter.max_angle_radians` were tuned to suppress noise from minor pose fluctuations. Through the fusion of filtered LiDAR data and IMU measurements, Cartographer achieved more reliable and drift-resistant SLAM performance. While LiDAR provided detailed spatial mapping through scan matching, the IMU contributed to maintaining accurate heading and motion estimation during situations where scan matching alone would have been less reliable.

3.5.2 Vision

The camera was utilised for two purposes: segmenting colours and detecting Aruco markers. The first was done using hue filtering, where the first component of the Hue-Saturation-Value (HSV) representation of the image was used to filter out colours, with the other two were configured to allow for a range of lighting conditions. Regions from the hue-filtered image were then further separated using common contours to detect specific objects in the scene. This was then implemented for two applications:

- following coloured objects in the environment. The minimum size threshold on the contour regions was set to remove noise and ensure only valid targets were processed. The robot was set to move left or right based on where the centre-point of the target was and move towards it until reaching a certain distance away from it, as demonstrated in Figure 5.
- visual servoing for the manipulator’s base joint to rotate and align itself with the object to be picked up. The manipulator’s end-effector was coloured distinctively to achieve tracking of both ends, and the operation is demonstrated in Figure 6.

3.5.3 Embedded systems

The ESP-32 were programmed with the ESP-IDF framework and the micro-ROS libraries to facilitate communication between it and the Pi. Our custom message `MecanumCmd` was compiled and we setup subscribers using it for commanding our wheels, as well as a publisher for the wheel encoder values. The transport we used was serial with the maximum baud-rate of 115200, as it provided less latency than anything wireless and was possible due to an abundance

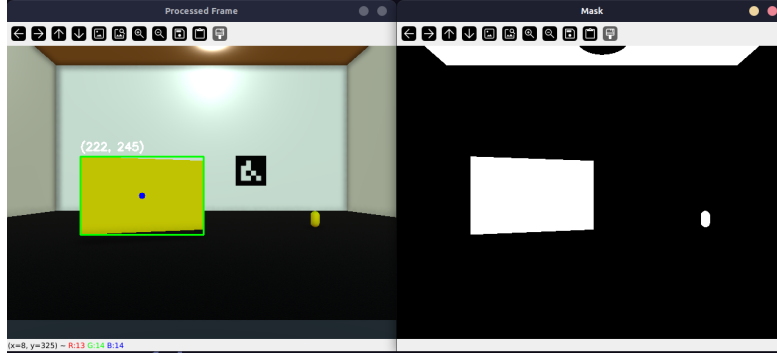


Figure 5: Yellow target detection

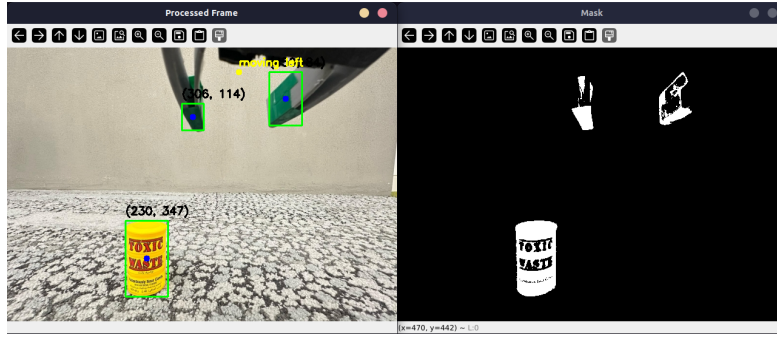


Figure 6: HSV mask and detection of object and gripper

of USB ports on the Pi.

3.5.4 Control

To command the mobile base, we computed the inverse kinematics (IK) of the robot to give us individual wheel velocities. These were then supplied as target velocities to a PID controller per-wheel, to ensure they were followed in stable manner. The IK equations are given as follows

$$\omega_{fl} = \frac{1}{r}(v_x - v_y - (l_x + l_y)\omega_z)$$

$$\omega_{fr} = \frac{1}{r}(v_x + v_y + (l_x + l_y)\omega_z)$$

$$\omega_{rl} = \frac{1}{r}(v_x + v_y - (l_x + l_y)\omega_z)$$

$$\omega_{rr} = \frac{1}{r}(v_x - v_y + (l_x + l_y)\omega_z)$$

3.5.5 Audio feedback

An audio playback service node was implemented to enable verbal interaction between the robot and its environment. The node, named `AudioService`, was developed to enable dynamic sound output on the robot through ROS 2 services. It provides two service endpoints: one to start audio playback and another to stop it.

This functionality allows the robot to respond to events or user interactions with verbal cues, alerts, or predefined audio messages and it is triggered when the robot detects a human or obstacle directly in front of it. Upon detection, the robot initiates audio playback for a specified duration before executing a maneuver to navigate around the object. This behaviour allows the robot to provide an audible cue such as a warning or instruction prior to acting, enhancing its interactive and socially aware operation. While this integration of the audio node into the robot’s feedback loop demonstrates effective modular design, it is acknowledged that the timing, responsiveness, and contextual awareness of this feature require refinement in the final prototype.

3.6 Circuit schematics

For the battery, we have used 4-cell 14.8V LiPo battery that gives power to motor driver “OSOYOO Model Y v2.0” and 4 motors. Then, from battery current goes to buck converter that steps it down from 14.8V to 5V.

The converter powers the Pi and I2C (PCA9685) board for the servo motors. Then Pi then powers Lidar Speaker. The simplified diagram presents an easier overview of the current flow in the system. The Raspberry Pi 5 also provides power for additional peripherals like the camera and LiDAR. The ESP32-S3 board handles the encoders to monitor motor speed and movement. The flowchart in Figure 7 helps visualize the high-level power distribution and the logical connections between the major components without diving into the detailed wiring. Figures 8 and 9 show the initial and final schematics of the wiring respectively, where each part is carefully labelled in the schematic to show how power and data are distributed between the motors, encoders, servo motors, sensors, and controllers.

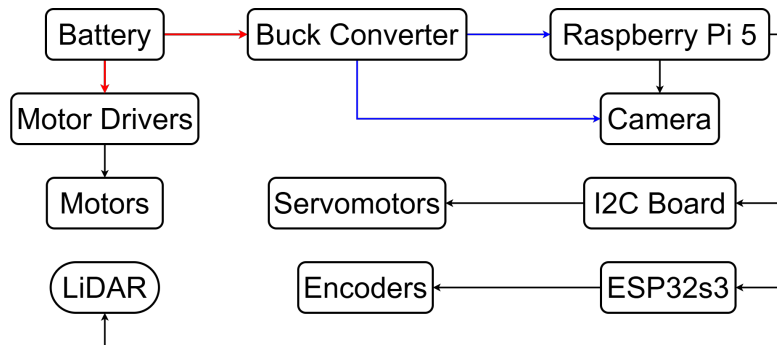


Figure 7: High-level electric circuit diagram

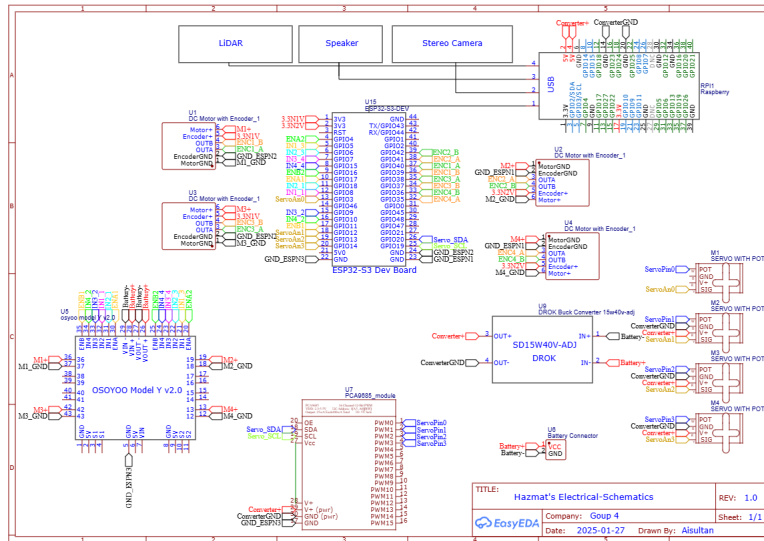


Figure 8: Previous schematics

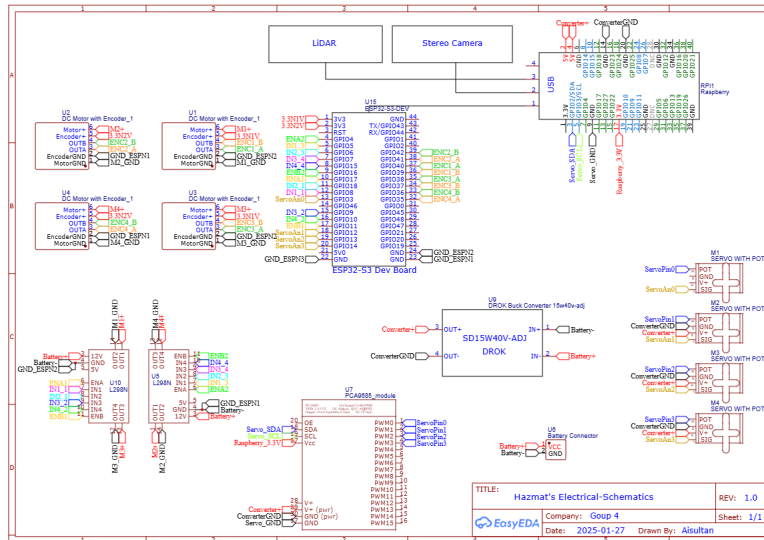


Figure 9: current schematics

3.7 Prototype development

4 Results

4.1 Computer simulation

We developed a full robot simulation in the Unity game engine with target points and Aruco markers in the scene to test and iterate on our navigation software in the initial stages of the project. The simulation was capable of simulating both RGB and depth frames for our stereo cameras, a LiDAR, an IMU, as wheels that can be individually commanded with feedback from encoders. The simulation environment can be seen in Figure 10.

4.2 Testing results

Testing our SLAM algorithm in real life, we were able to map and localise in it later with high accuracy, the produced map can be seen in Figure 11. The odometry node was able to localise direction quite accurately too, with noticeable drift beyond 2 metres of movement.

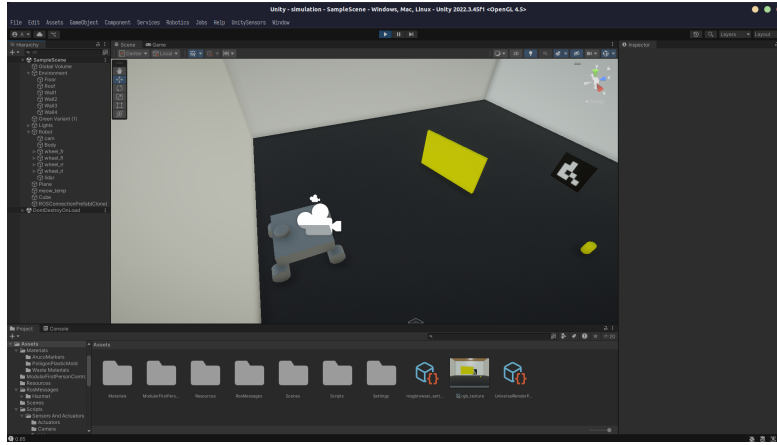


Figure 10: Simulation environment inside the Unity editor

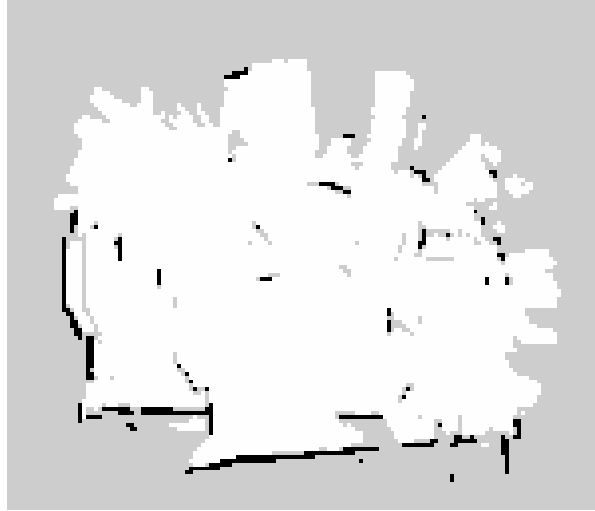


Figure 11: 2D occupancy map of the East end of the EE lab

Our autonomous navigation logic allowed the robot to reach its required target positions with a tolerance of ± 3 cm due to delay in getting a LiDAR reading to actually commanding the robot. For the manipulator, we required manual placement of objects away from the robot, as our logic mainly only changed the rotation of the first joint to track the object.

5 Management and Marketing

5.1 Project Management

5.2 Project Finance

5.3 Marketing

5.4 Risk Management

5.5 Health, Safety, and Environment

6 Conclusion and Future Work

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074. [Online]. Available: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.